King Fahd University of Petroleum and Minerals
College of Computing and Mathematics

Information & Computer Science Department



ICS 202
Data Structures and Algorithms

Unit 14

Data Compression

Term 252

Reading Assignment

- Data Structures and Algorithms in Java, 4th Edition, Adam Drozdek, Cengage Learning, ISBN 978-9814239233
  - Chapter 11
    - Section 11.2: Huffman Coding (11.2.1 is not included)
    - Section 11.4: Ziv-Lempel Code (LZW is not included)

Data Compression and Huffman Coding

- Introduction
  - What is Data Compression?
  - Why Data Compression?
- Lossless and Lossy Data Compression
  - Static, Adaptive, and Hybrid Compression
- Compression Techniques
  - Static Huffman Coding
  - Lempel-Ziv Encoding

What is Data Compression?

Representing information in a compact form

- What is Data Compression?
  - The representation of information using fewer bits than the original representation, while (ideally) preserving the essential content
  - Representation of information as accurately as possible using the fewest number of bits.

- Why Data Compression?
  - Save storage space (smaller files)
  - Speed up transmission over networks
  - Reduce costs (bandwidth, storage)
  - Improve performance (faster loading times)

- Real-World Examples:
  - ZIP, RAR, 7z ⟶ compressed archives.
  - JPEG, MP3, MP4 ⟶ for images, audio, and video.
  - PNG, FLAC ⟶ formats preserving original quality.

Types of Data Compression

Can Original data be perfectly recovered ?

	Lossless Compression	Lossy Compression
Recovered data	No loss, decompressed data is identical to the original	decompressed data is not identical to the original but close enough for human perception
How it works?	Removes redundancy	Removes less important data (and redundancy) based on perceptual models
Use Cases	Text files, databases	Images, audio, and video
Common Algorithms	Run-length, Huffman Coding, LZW, LZ77, LZ78	Images (JPEG), Audio (MP3), Video (MPEG)

- Lossy techniques: higher compression rates
- Lossless techniques: more accurate
  - An uncompressed image: 10 MB. Using PNG: 7 MB no quality loss, Using JPEG: 500 KB with minor visual quality reduction

Classification of Lossless Compression Techniques

- **Static compression:**
  - Sender builds a model (e.g., Huffman tree)
  - Sender compresses data using that model
  - Sender transmits:
    - the model (tree / codebook)
    - the compressed bitstream
  - Receiver:
    - reads the model first
    - then decodes the message

- Mapping from messages to codewords is fixed before transmission.
- Each message corresponds to the same codeword every time it appears in the encoded message.
- Predictable encoding for a given message.
- Requires two passes:
  - first pass to compute frequency and determine the mapping, and a second pass to encode.
- Examples: Static Huffman Coding

- **Dynamic (adaptive) methods:**
  - do NOT send a model (full tree) upfront
  - both sides build it gradually

- The mapping from messages to codewords changes dynamically over time.
- Only one scan of the message is needed for the entire encoding process.
- Examples: LZ77, LZ78, LZW, and Adaptive Huffman Coding

Compression Techniques: Static Huffman Coding

- Static Huffman coding assigns variable length codes to symbols (such as characters) based on their frequency of occurrences in the given message.
  - high frequency symbols are assigned shorter codes; fewer bits
  - low frequency symbols are assigned longer codes; many bits.
  - How It Works:

	Phase	Description
1	Frequency Analysis	Scan the entire message and count how many times each symbol appears.
2	Building the Huffman Tree	Using the frequencies, generates a binary tree, the Huffman code tree, branches labeled with (0 and 1).
3	Generating Codes	Traverse the tree from root to each leaf. The sequence of 0s and 1s along the path becomes that symbol's code.
4	Encoding	Replace every symbol in the input message with its Huffman code
5	Store/Transmit	The compressed bitstream is sent/stored along with the code table

Static Huffman Coding Algorithm

- Find the frequency of each character in the file to be compressed;
- For each distinct character create a one-node binary tree containing the character and its frequency as its priority;
  - Insert the one-node binary trees in a priority queue in increasing order of frequency;

```
while (there exists more than one tree in the priority queue) {
        dequeue two trees t1 and t2;
        Create a tree t that contains t1 as its left subtree and t2 as its
        right subtree;  // 1
        priority (t) = priority(t1) + priority(t2);
        insert t in its proper location in the priority queue;  // 2
 }
```

- Assign 0 and 1 weights to the edges of the resulting tree, such that the left and right edge of each node do not have the same weight; // 3
- Notes:
  - This is an example of a greedy algorithm.
  - The Huffman code tree for a particular set of characters is not unique (Steps 1, 2, and 3 may be done differently).

```
function BUILD_HUFFMAN_TREE(freq):
    create a min-heap H
    for each symbol s in freq:
        create a leaf node with weight freq[s]
        insert node into H

    while size(H) > 1:
        left  = extractMin(H)
        right = extractMin(H)
        parent = new node with weight left.weight + right.weight
        parent.left  = left
        parent.right = right
        insert parent into H

    return extractMin(H)    // the root of the Huffman tree
```

```
function GENERATE_CODES(node, prefix, codeTable):
    if node is a leaf:
        codeTable[node.symbol] = prefix
        return

    GENERATE_CODES(node.left,  prefix + "0", codeTable)
    GENERATE_CODES(node.right, prefix + "1", codeTable)
```

```
function ENCODE(message, codeTable):
    encoded = ""
    for each symbol s in message:
        encoded = encoded + codeTable[s]
    return encoded
```

Static Huffman Coding example

- Example: Information to be transmitted over the internet contains the following characters with their associated frequencies:

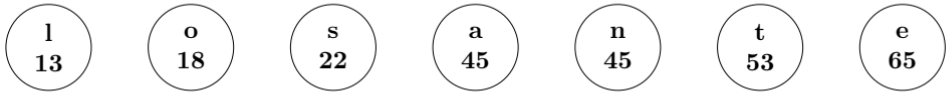
Character	a	e	l	n	o	s	t
Frequency	45	65	13	45	18	22	53

- Build the Huffman code tree for the message.
- Use the Huffman tree to find the codeword for each character.
- If the data consists of only these characters, what is the total number of bits to be transmitted? What is the compression ratio?
- Verify that your computed Huffman codewords satisfy the Prefix property.

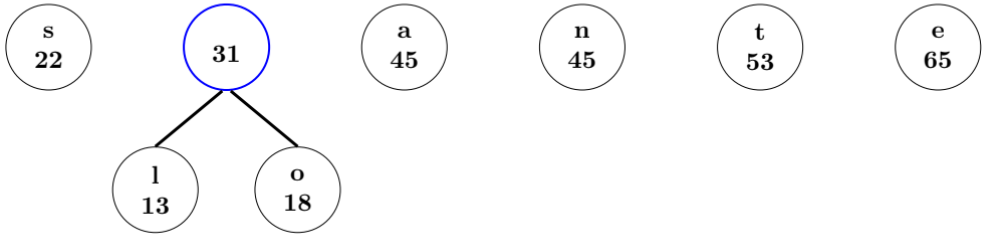
Static Huffman Coding: Building the tree

Character	frequency
a	45
e	65
l	13
n	45
o	18
s	22
t	53

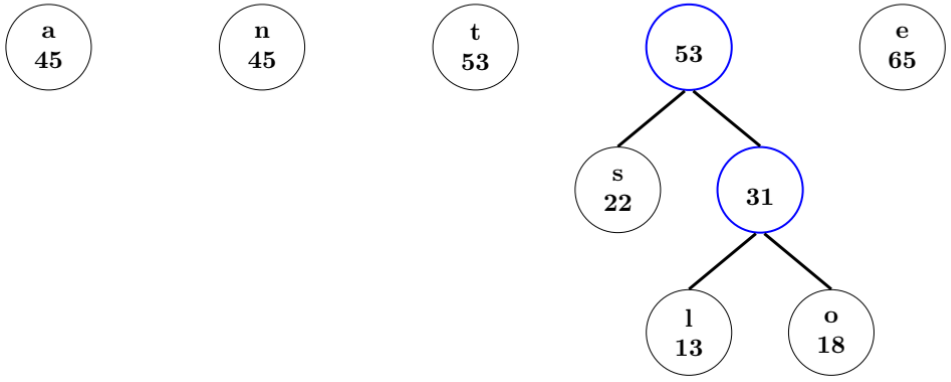
Step 0 — Initial sorted nodes



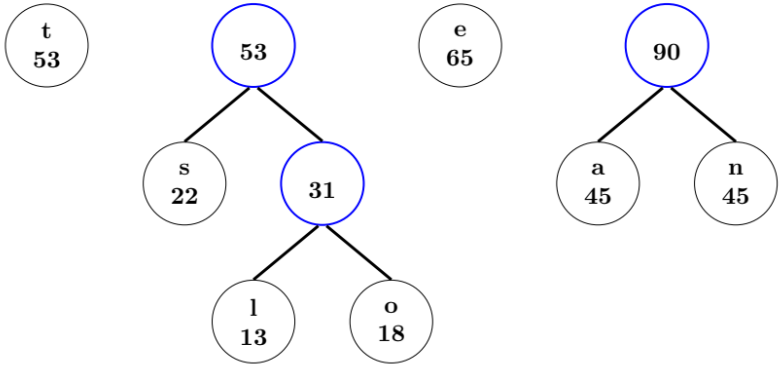
Step 1 — merge l(13) + o(18) → 31



Step 2 — merge s(22) + 31 → 53

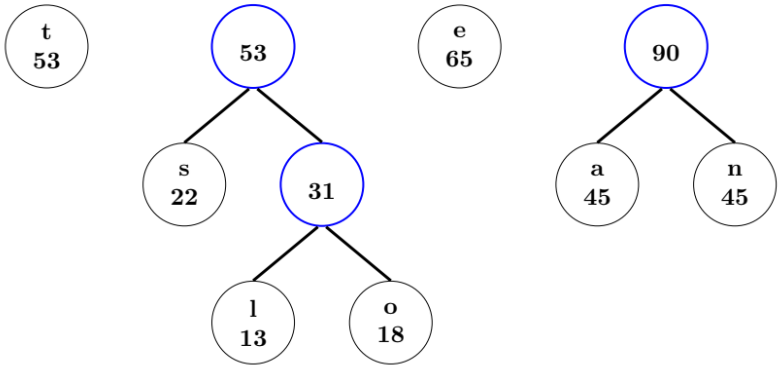


Step 3 — merge a(45) + n(45) → 90

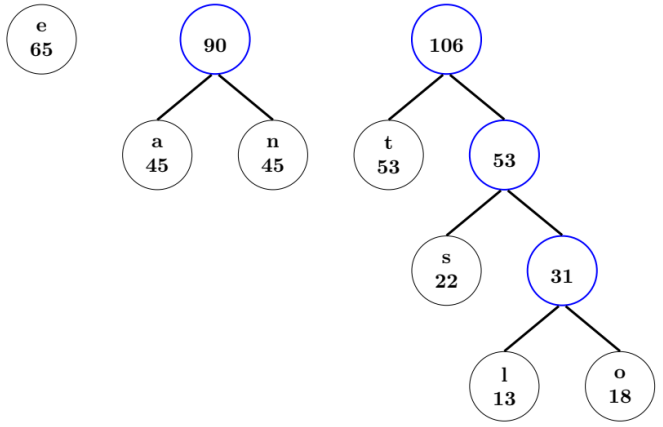


Static Huffman Coding: Building the tree

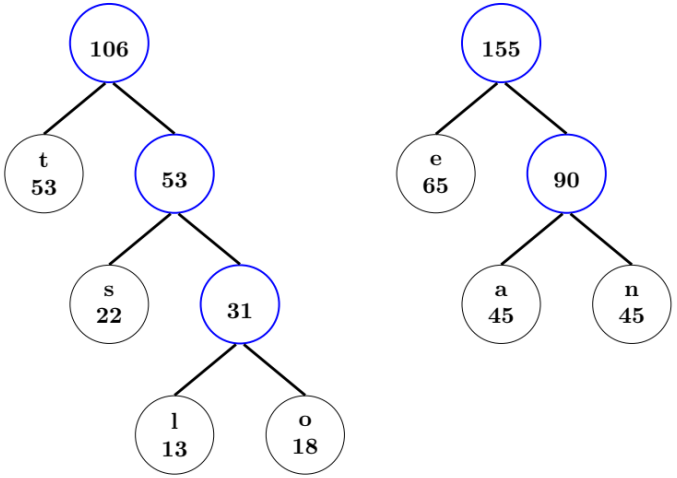
Step 3 — merge a(45) + n(45) → 90



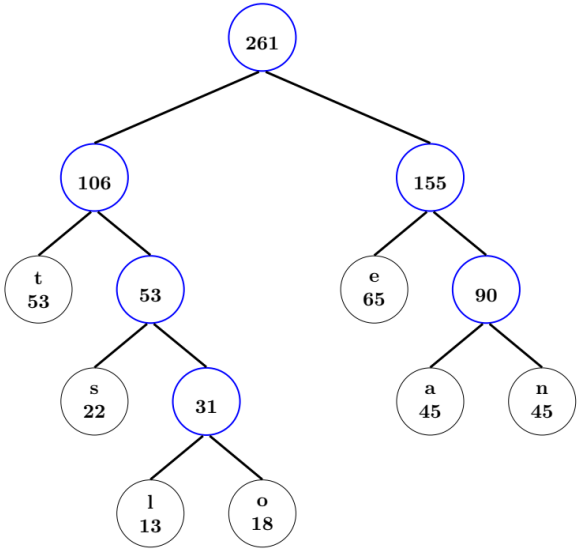
Step 4 — merge t(53) + 53 → 106



Step 5 — merge e(65) + 90 → 155

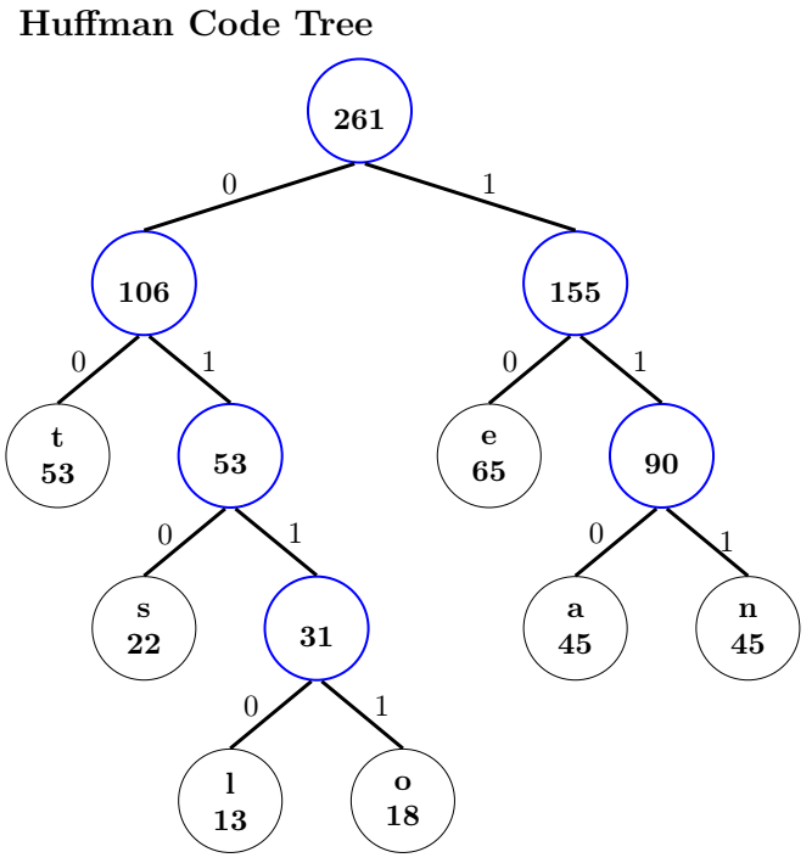


Step 6 — Final merge: 155 + 106 → 261



- **Huffman always:**
  - picks **two smallest**
  - builds bottom-up
  - uses greedy strategy

Static Huffman Coding: Generating codes



character	a	e	l	n	o	s	t
codeword	110	10	0110	111	0111	010	00

The sequence of zeros and ones that are the arcs in the path from the root to each leaf node are the desired codes:

```
function GENERATE_CODES(node, prefix, codeTable):
    if node is a leaf:
        codeTable[node.symbol] = prefix
        return

    GENERATE_CODES(node.left,  prefix + "0", codeTable)
    GENERATE_CODES(node.right, prefix + "1", codeTable)
```

- Encode (compress) the message **tenseas** using the following codewords:
- Answer: Replace each character with its codeword:
  - **tenseas**
  - **00101111010101100010**

```
function ENCODE(message, codeTable):
    encoded = ""
    for each symbol s in message:
        encoded = encoded + codeTable[s]
    return encoded
```

TRY: https://opendsa-server.cs.vt.edu/ODSA/Books/Everything/html/Huffman.html

Decoding

- Decode (decompress) each of the encoded messages 0110011101000 and 11101110101011, if possible, using the Huffman codeword given tree

Answer: Decode a bit-stream by

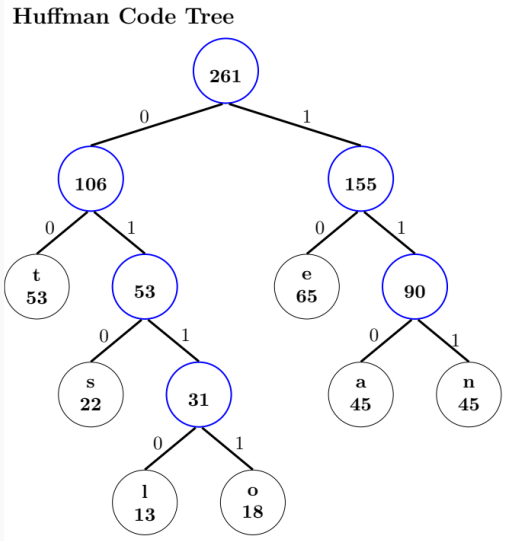
- Start at the root
- Navigate down the tree according to the bits in the message (0 = left, 1 = right).
- When a leaf is encountered,
  - output the character at that leaf
  - restart at the root.
- If a leaf cannot be reached, the bit-stream cannot be decoded.

```
function DECODE(bitstream, root):
    output = ""
    current = root

    for each bit b in bitstream:
        if b == '0':
            current = current.left
        else:
            current = current.right

        if current is a leaf:
            output = output + current.symbol
            current = root

    return output
```



a) 0110011101000 => "lost"

b) 11101110101011 ← The decoding fails because the corresponding node for 11 is not a leaf

character	a	e	l	n	o	s	t
Huffman codeword	110	10	0110	111	0111	010	00

The Prefix Property

- Data encoded using Huffman coding is uniquely decodable.
  - This is because Huffman codes satisfy an important property called the prefix property:
  - In each set of Huffman codewords, no codeword is a prefix of another
  - For example, in a given set of Huffman codewords, 10 and 101 cannot simultaneously be valid Huffman codewords because the first, 10, is a prefix of the second, 101.

- To see why the prefix property is essential, consider the codewords given below in which "e" is encoded with 110 which is a prefix of "f"

- The decoding of 11000100110 is ambiguous:
  - **11000100110** => **face**
  - **11000100110** => **eaace**

character	a	b	c	d	e	f
codeword	0	101	100	111	110	1100

Transmitting the Huffman tree

- To store the Huffman tree at the beginning of the compressed file,
  - a post-order traversal is used, writing each node visited.
  - When a leaf node is encountered, a 1-bit is written followed by the 8-bit ASCII code of the leaf node character.
  - When a non-leaf node is encountered, a 0-bit is written.
  - To indicate the end of the Huffman coding tree, another 0-bit is written.

```
procedure EncodeHuffmanTree(root):
    output = empty string
    PostOrderWrite(root, output)
    append "0" to output        // end-of
    return output


procedure PostOrderWrite(node, output):

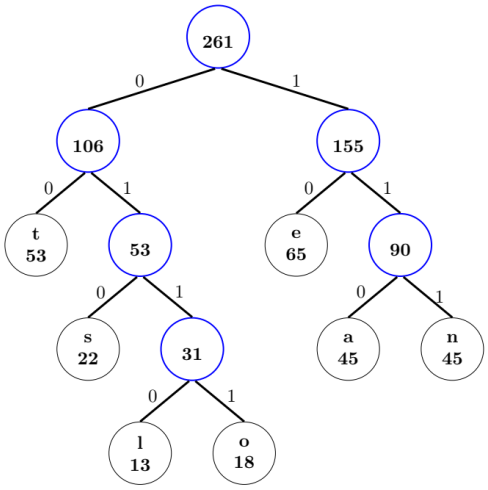
    if node is NULL:
        return

    // Traverse left subtree
    PostOrderWrite(node.left, output)

    // Traverse right subtree
    PostOrderWrite(node.right, output)

    // Process the current node
    if node is a leaf:
        append "1" to output
        append ASCII(node.symbol) to outp
    else:
        append "0" to output
```

Huffman Code Tree



- Example: The Huffman tree shown below can be transmitted as:

  1T1S1L1O0001E1A1N0000

- Replacing each character by its 8-bit ASCII code, we have:

  10111010010111001110110110010110111100010110010110110000110110111000 00

- Number of bits to transmit the tree = 7 * 8 + 14 = 70

Static Huffman Coding example (cont'd)



- If we assume the message consists of only the characters A,E,L,N,O,S,T then the number of bits for the compressed message will be 696:

Character	A	E	L	N	O	S	T	Total
Codeword	110	10	0110	111	0111	010	00	
Codeword bits	3	2	4	3	4	3	2	
Char. Frequency	45	65	13	45	18	22	53	261
Codeword bits * Char. Frequency	135	130	52	135	72	66	106	696

- The total number of bits to transmit the message including the Huffman tree is 696 + 70 = 766
  - If the message is sent uncompressed with 8-bit ASCII for the characters, we have 261*8 = 2088 bits.
- Compression ratio = bits before compression / bits after compression = $\frac{2088}{766} \approx 2.72$
  - For every 2.72 MB (or any unit) of original data, it becomes 1 MB after compression.
- Thus, the size of the compressed file is $\frac{766}{2088} \approx 36.68\ \%$ of the original ASCII file.

Recovering the Huffman tree

- To recover the transmitted Huffman tree, the receiver uses a stack.
- When a 1-bit is read,
  - the next byte is read and the corresponding ASCII character is pushed onto the stack.
- When a 0-bit is read,

  if the stack contains only one element,

    the entire Huffman code tree has been constructed.

  else // there must be more than one element in the stack.

    A new node is created with two children as follows:

    The top two elements are popped off the stack;

    The first element becomes the right child; and

    The second element becomes the left child;

    The newly created node is pushed onto the stack.

```
procedure DecodeHuffmanTree(bitstream):

    stack = empty stack

    while bitstream has more bits:
        bit = read next bit

        if bit == '1':
            c = read next 8 bits                // character
            push new leaf node(c) onto stack

        else if bit == '0' and more bits exist:
            // internal node: pop right and left children
            right = pop stack
            left = pop stack
            node = new internal node(left, right)
            push node onto stack

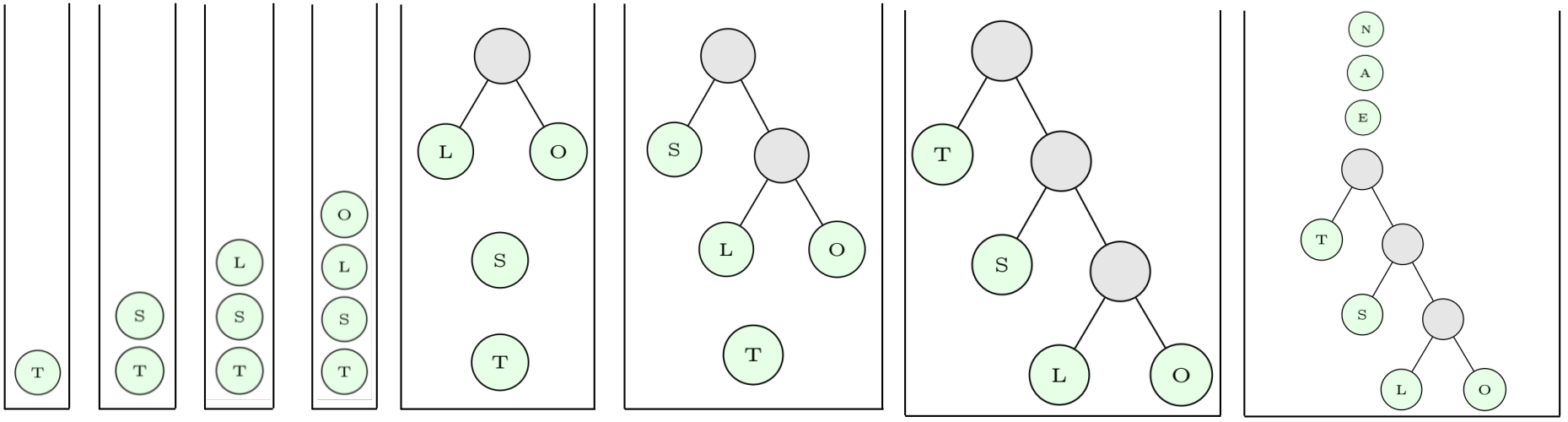
        else if bit == '0' and no more bits:
            break   // end-of-tree marker

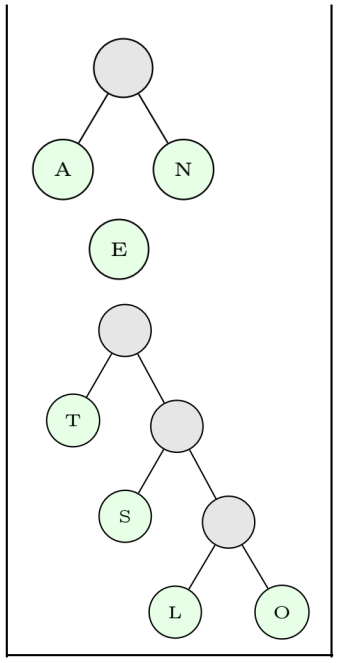
    return pop stack    // root of reconstructed tree
```

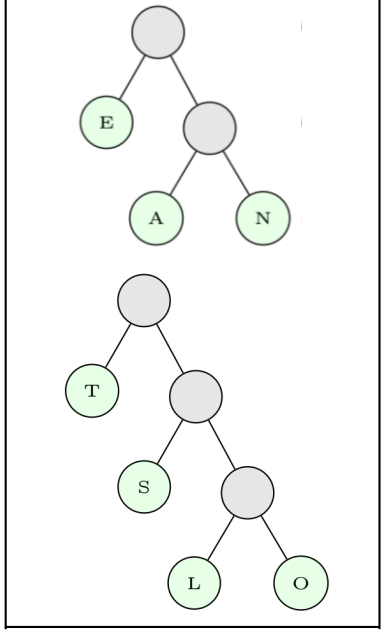
1: PUSH the next character/byte.

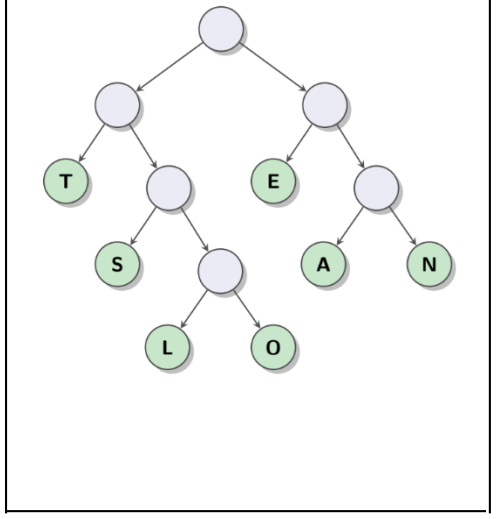
0: POP 2 nodes, join them by creating a parent node and push the parent node

Recovering the Huffman tree: 1T1S1L1O0001E1A1N0000

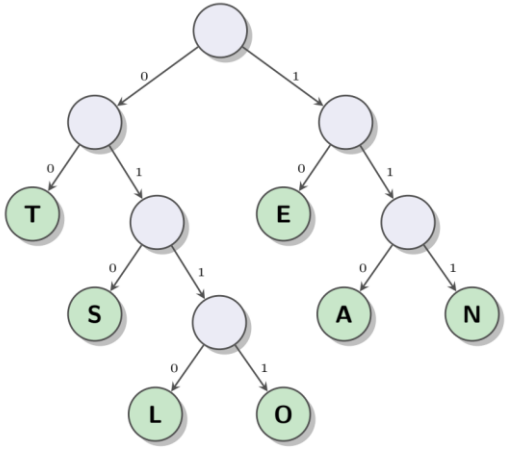








Note: It is only the addresses of root nodes that are pushed onto the stack and not entire subtrees.



LZ78 Compression Algorithm

- LZ78 is a dictionary-based lossless compression algorithm invented by Lempel & Ziv (1978).

- LZ78 inserts one- or multi-character, non-overlapping, distinct patterns of the message to be encoded in a Dictionary. i.e. (index, pattern)

  - The multi-character patterns are of the form: $C_0C_1 \ldots C_{n-1}C_n$. The prefix of a pattern consists of all the pattern characters except the last: $C_0C_1 \ldots C_{n-1}C_{n-1}$

- LZ78:

Output	Condition
(0, c)	if a one-character pattern c is not in the dictionary
(DictionaryPrefixIndex, c_n)	if a multi-character pattern $c_1c_2c_3 \ldots c_{n-1}c_n$ is not in the dictionary
(DictionaryPrefixIndex,)	if the last input character c_n or the last multi-character pattern p_n is in the dictionary. $p_1p_2p_3 \ldots p_{n-1}c_n$ or $p_1p_2p_3 \ldots p_{n-1}p_n$

- Note: The dictionary is usually implemented as a hash table.

LZ78 Compression Algorithm

- Idea: Replace repeated phrases in the input with (index, next character) pairs.

```
Initialize dictionary with empty string and index 0
p ⟵ "" // current prefix
while there are more input characters do
        k ⟵ read next character
        pK ⟵ p + k
        if (pK) exists in dictionary then
                p ⟵ pK // extend the prefix
        else
                w ⟵ index of p
                output (w, k)
                add (pK) to dictionary
                p ⟵ ""
        end if
end while
if p != "" then
        w ⟵ index of p
        output (w,  )
end if
```

```java
public static List<Pair> encode(String input) {
    Map<String, Integer> dictionary = new HashMap<>();
    dictionary.put( "",0);
    List<Pair> output = new ArrayList<>();

    String p = "";
    int dictIndex = 1; // start indexing from 1

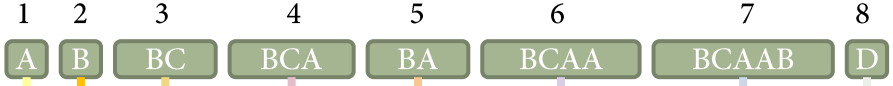
    for (int i = 0; i < input.length(); i++) {
        char k = input.charAt(i);
        String pk = p + k;

        if (dictionary.containsKey(pk)) {
            p = pk; // extend prefix
        } else {
            //int w = p.isEmpty() ? 0 : dictionary.get(p);
            int w =  dictionary.get(p);
            output.add(new Pair(w, k));

            dictionary.put(pk, dictIndex++);
            p = ""; // reset prefix
        }
    }
    // handle leftover prefix
    if (!p.isEmpty()) {
        int w = dictionary.get(p);
        output.add(new Pair(w, '\0')); // null character
    }
```

Example 1: LZ78 Compression

- Encode the string ABBCBCABABCAABCAABD



Dictionary

Output	Index	Pattern
(0, A)	1	A
(0, B)	2	B
(2, C)	3	BC
(3, A)	4	BCA
(2, A)	5	BA
(4, A)	6	BCAA
(6, B)	7	BCAAB
(0, D)	8	D

1. **A** is not in the Dictionary; insert it
2. **B** is not in the Dictionary; insert it
3. **B** is in the Dictionary, extend
   BC is not in the Dictionary; insert it.
4. **B** is in the Dictionary, extend
   BC is in the Dictionary, extend
   BCA is not in the Dictionary; insert it.
5. **B** is in the Dictionary, extend
   BA is not in the Dictionary; insert it.
6. **B** is in the Dictionary, extend
   BC is in the Dictionary, extend
   BCA is in the Dictionary, extend
   BCAA is not in the Dictionary; insert it.
7. **B** is in the Dictionary, extend
   BC is in the Dictionary, extend
   BCA is in the Dictionary, extend
   BCAA is in the Dictionary, extend
   BCAAB is not in the Dictionary; insert it.
8. **D** is not in the Dictionary; insert it

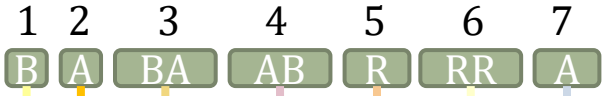
The compressed message is:

(0, A)(0, B)(2, C)(3, A)(2, A)(4, A)(6, B)(0, D)

0A0B2C3A2A4A6B0D

Example 2: LZ78 Compression

- Encode (i.e., compress) the string BABAABRRRA using the LZ78 algorithm.



Dictionary

Output	Index	Pattern
(0, B)	1	B
(0, A)	2	A
(1, A)	3	BA
(2, B)	4	AB
(0, R)	5	R
(5, R)	6	RR
(2,)		

1. **B** is not in the Dictionary; insert it
2. **A** is not in the Dictionary; insert it
3. **B** is in the Dictionary , extend
   BA is not in the Dictionary; insert it
4. **A** is in the Dictionary , extend
   AB is not in the Dictionary; insert it
5. **R** is not in the Dictionary; insert it
6. **R** is in the Dictionary , extend
   RR is not in the Dictionary; insert it
7. **A** is in the Dictionary and it is the last input character; output a pair containing its index: **(2,)**

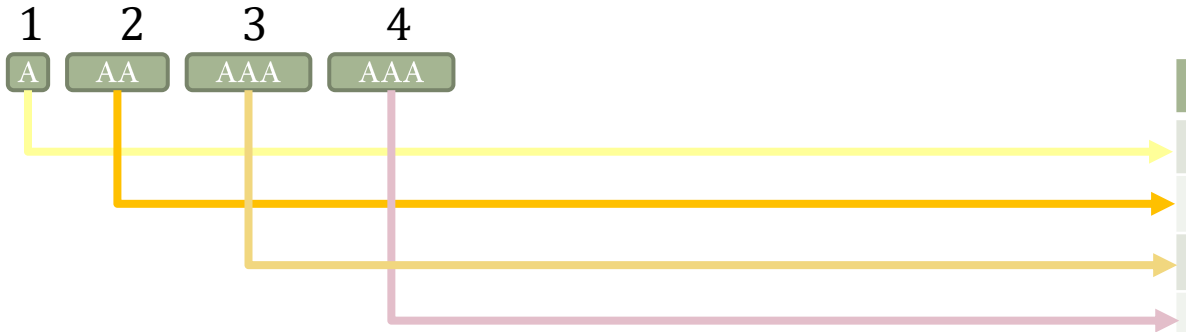
The compressed message is:

(0, B)(0, A)(1, A)(2, B)(0, R)(5, R)(2,)

0B0A1A2B0R5R2

Example 3: LZ78 Compression

- Encode the string AAAAAAAAAA using the LZ78 algorithm.



Output
(0, A)
(1, A)
(2, A)
(3,)

Dictionary

Index	Pattern
1	A
2	AA
3	AAA

1. A is not in the Dictionary; insert it
2. A is in the Dictionary , extend
   AA is not in the Dictionary; insert it
3. A is in the Dictionary , extend
   AA is in the Dictionary
   AAA is not in the Dictionary; insert it
4. A is in the Dictionary , extend
   AA is in the Dictionary , extend
   AAA is in the Dictionary, and it is the last pattern; output a pair containing its index: (3,)

Self Study

- The compressed message is: (0, A)(1, A)(2, A)(3,)
  - 0A1A2A3
- Note: The above is just a representation, the commas and parentheses are not transmitted

LZ78 Compression: Number of bits transmitted

- Example: Uncompressed String: ABBCBCABABCAABCAABD

  Number of bits = Total number of characters * 8

  = 19 * 8 = 152 bits

- Suppose the codewords are indexed starting from 1:
  - Compressed string(codewords): (0, A) (0, B) (2, C) (3, A) (2, A) (4, A) (6, B) (0, D)
  - Codeword index 1 2 3 4 5 6 7 8
- Each code word consists of an integer and a character:
  - The character is represented by 8 bits.
  - The number of bits n required to represent the integer part of the codeword with index i is given by:

$$n = \begin{cases} 1 & \text{if } i = 1 \\ \lceil \log_2 i \rceil & \text{if } i > 1 \end{cases}$$

- Alternatively, the number of bits required to represent the integer part of the codeword with index i is the number of significant bits required to represent the integer i – 1

LZ78 Compression: Number of bits transmitted (cont'd)

index	index-1	bits	Number of significant bits
1	0	0	1
2	1	1	
3	2	10	2
4	3	11	
5	4	100	3
6	5	101	
7	6	110	
8	7	111	
9	8	1000	4
10	9	1001	
11	10	1010	
12	11	1011	
13	12	1100	
14	13	1101	
15	14	1110	
16	15	1111	

Codeword (0, A) (0, B) (2, C) (3, A) (2, A) (4, A) (6, B) (0, D)

index 1 2 3 4 5 6 7 8

Bits: (1+8) + (1+8) + (2+8) + (2+8) + (3+8) + (3+8) + (3+8) + (3+8) = 82 bits

- The actual compressed message is: 0A0B10C11A010A100A110B000D where each character is replaced by its binary 8-bit ASCII code.
- To represent (0,D), we need to represent the character D and the integer 0.
  - D is represented using 8 bits.
  - The index of (0,D) is 8. 8-1 is 7. 7 needs 3 bits so we have to represent 0 using 3 bits as 000
- To represent (2,A). we need to represent the character A and the integer 2.
  - A is represented using 8 bits
  - (2,A) index is 5 . 5-1 = 4. 4 needs 3 bits so 2 will be represented as 010
- ANOTHER WAY: Represent the index as 2^k. Ceil(k) is the number of bits.
  - (2,A) index is 5 . $5 = 2^x \longrightarrow x = \lceil \log_2 5 \rceil = 3$.
- Compression ratio = bits before compression / bits after compression =19* 8 / 82

Index	1	2	3	4	5	6	7	8	9	…	16
Written as		2^1		2^2				2^3		…	2^4
#Bits	1	1		2				3	4	…	4

LZ78 Decompression Algorithm

```
Algorithm LZ78_Decompress(codeStream)

    Dictionary ← empty
    DictionaryIndex ← 1
    OutputText ← empty

    while (codeStream is not empty) do
        (Index, Char) ← next codeword from codeStream

        if (Index = 0) then
            // Char is a new standalone symbol
            Entry ← Char
        else
            // Retrieve prefix from dictionary
            Entry ← Dictionary[Index] + Char
        end if

        Append Entry to OutputText

        // Insert new entry in dictionary
        Dictionary[DictionaryIndex] ← Entry
        DictionaryIndex ← DictionaryIndex + 1
    end while

    return OutputText

End Algorithm
```

Summary:
input: (index, character) pairs

if(index == 0)
    output: character
else
    output: stringAtIndex(index) + character

insert current output in dictionary

```java
// DECODE
public static String decode(List<Pair> encoded) {
    Map<Integer, String> dictionary = new HashMap<>();
    StringBuilder result = new StringBuilder();

    int dictIndex = 1;

    for (Pair pair : encoded) {
        String entry;

        if (pair.index == 0) {
            entry = (pair.symbol == '\0') ? "" :
                    String.valueOf(pair.symbol);
        } else {
            entry = dictionary.get(pair.index);
            if (pair.symbol != '\0') {
                entry = entry + pair.symbol;
            }
        }

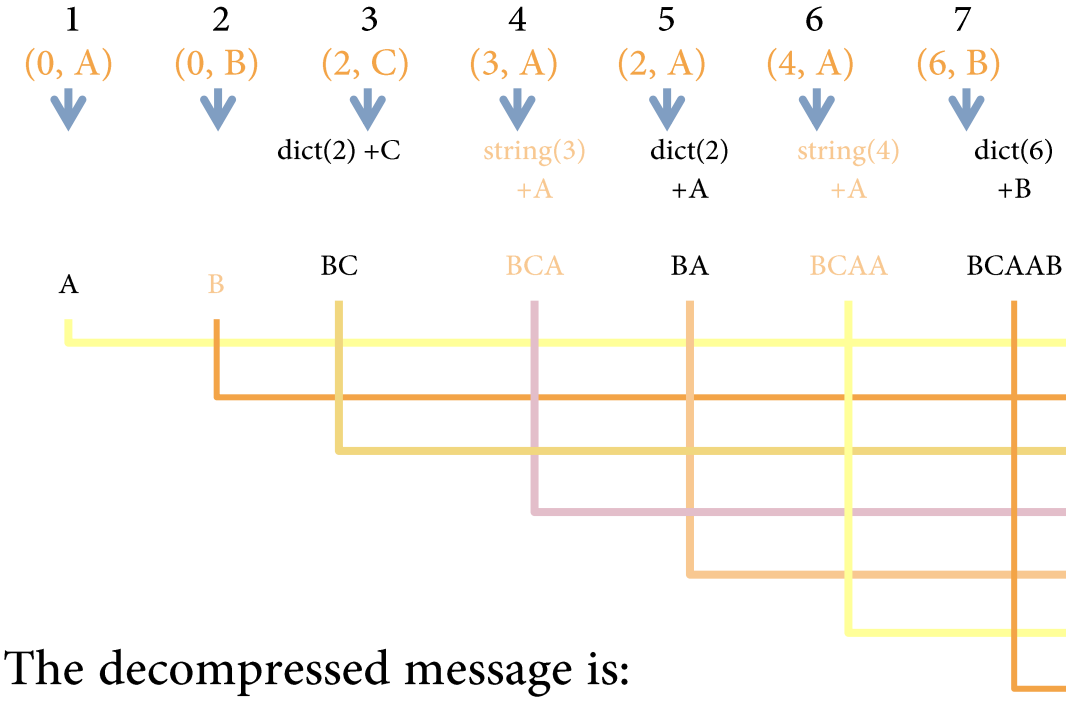
        result.append(entry);

        if (!entry.isEmpty()) {
            dictionary.put(dictIndex++, entry);
        }
    }

    return result.toString();
```

Example: LZ78 Decompression

- Decode (i.e., decompress) the sequence: (0, A) (0, B) (2, C) (3, A) (2, A) (4, A) (6, B)



Output
A
B
BC
BCA
BA
BCAA
BCAAB

Dictionary

Index	Pattern
1	A
2	B
3	BC
4	BCA
5	BA
6	BCAA
7	BCAAB

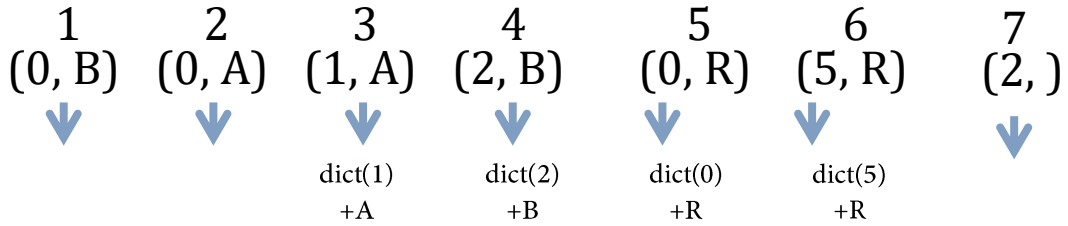
- The decompressed message is:
  - ABBCBCABABCAABCAAB ; length =18
- Compression ratio = bits before compression / bits after compression =

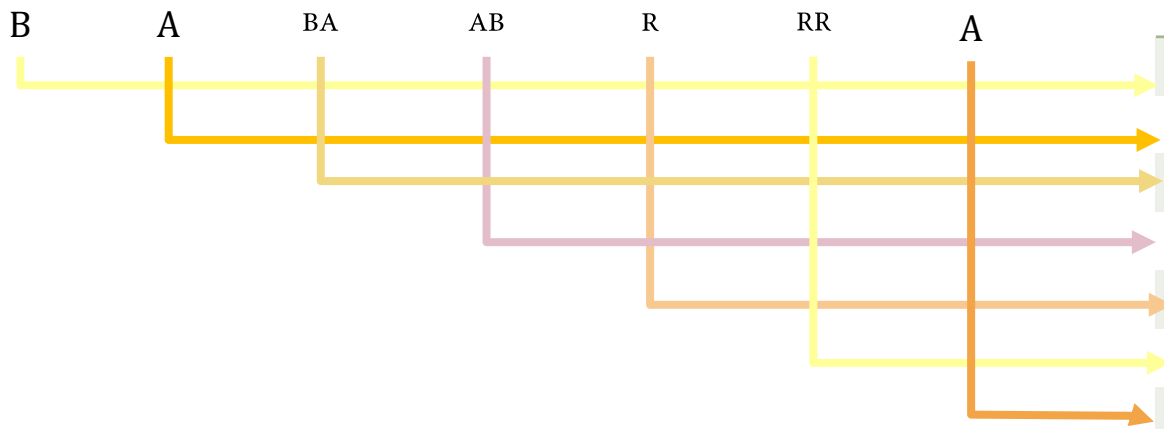
  18* 8 / 71

Codeword	(0, A)	(0, B)	(2, C)	(3, A)	(2, A)	(4, A)	(6, B)	
index	1	2	3	4	5	6	7	
Bits:	(1+8) +	(1+8) +	(2+8) +	(2+8) +	(3+8) +	(3+8) +	(3+8)	= 71 bits

Example: LZ78 Decompression

- Decode the sequence: (0, B) (0, A) (1, A) (2, B) (0, R) (5, R) (2,)





Dictionary

Output	Index	Pattern
B	1	B
A	2	A
BA	3	BA
AB	4	AB
R	5	R
RR	6	RR
A		

- The decompressed message is: BABAABRRRA
- Compression ratio = bits before compression / bits after compression = 10* 8 / 63

Codeword	(0, B)	(0, A)	(1, A)	(2, B)	(0, R)	(5, R)	(2,)
index	1	2	3	4	5	6	7
Bits:	(1+8) +	(1+8) +	(2+8) +	(2+8) +	(3+8) +	(3+8) +	(3) = 63 bits

Example 3: LZ78 Decompression

- Decode (i.e., decompress) the sequence

	1	2	3	4
	(0, A)	(1, A)	(2, A)	(3,)
	A	AA	AAA	AAA

Dictionary

Output	Index	Pattern
A	1	A
AA	2	AA
AAA	3	AAA
AAA		

The decompressed message is: AAAAAAAAA

Exercises

- Using the Huffman tree constructed in this session, decode the following sequence of bits, if possible. Otherwise, where does the decoding fail? 10100010111010001000010011
- Using the Huffman tree constructed in this session, write the bit sequences that encode the messages: test , state , telnet , notes
- Mention one disadvantage of a lossless compression scheme and one disadvantage of a lossy compression scheme.
- Write a Java program that implements the Huffman coding algorithm.
- Use LZ78 to trace encoding the string SATATASACITASA.
- Write a Java program that encodes a given string using LZ78.
- Write a Java program that decodes a given set of encoded codewords using LZ78.